



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Course Code: CSE 402.

Course Title: Operating System Lab.

Experiment No: 1

Experiment Title: Implementation of First Come First Serve (FCFS)
CPU Scheduling Algorithm.

Date of Submission : 02-08-2026

Submitted By:

Name : Boishakhi Basu.

Student Id : 22201173

Semester : 4th Year 1st Semester

Section : C-2

Submitted To
Aita Rahman Orthi
Lecturer , UAP

Problem Statement:

Implement FCFS CPU Scheduling for the following scenario where No of processes = 5 and Process informations are-

P id	AT	BT
P0	3	1
P1	5	3
P2	2	2
P3	1	2
P4	6	3

Calculate CT, TAT, WT, AVG TAT and AVG WT. Also represent the process execution sequence.

Source Code:

```
22201173.py X
C: > Users > hp > Desktop > New folder > 22201173.py

1  processes = [
2      ["P0", 3, 1],
3      ["P1", 5, 3],
4      ["P2", 2, 2],
5      ["P3", 1, 2],
6      ["P4", 6, 3]
7  ]
8
9  processes.sort(key=lambda x: x[1])
10
11  time = 0
12  total_tat = 0
13  total_wt = 0
14
15  print("PID\tAT\tBT\tCT\tTAT\tWT")
16
```

```

17 for pid, at, bt in processes:
18     if time < at:
19         time = at
20
21     time = time + bt
22     ct = time
23
24     tat = ct - at
25
26     wt = tat - bt
27
28     total_tat += tat
29     total_wt += wt
30
31     print(pid, "\t", at, "\t", bt, "\t", ct, "\t", tat, "\t", wt)
32
33     ava_tat = total_tat / len(processes)
34     ava_wt = total_wt / len(processes)
35
36     print("\nAverage TAT =", ava_tat)
37     print("Average WT =", ava_wt)

```

Output:

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
PS C:\Users\hp\Desktop\New folder> & "C:\Program File				
PID	AT	BT	CT	AT
P3	1	2	3	2
P2	2	2	5	3
P0	3	1	6	3
P1	5	3	9	4
P4	6	3	12	6
Average TAT = 3.6				
Average WT = 1.4				
PS C:\Users\hp\Desktop\New folder> █				

Discussion:

In this experiment, the First Come First Serve (FCFS) CPU Scheduling algorithm was implemented using Python. The processes were initially stored in a list containing the Process ID, Arrival Time (AT), and Burst Time (BT). Before scheduling, the processes were sorted according to their arrival times so that they could be executed in the correct FCFS order.

During execution, the program checked whether the CPU was idle before the arrival of a process. If the current CPU time was less than the process's arrival time, the CPU remained idle until that process arrived. After each process completed its execution, the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) were calculated using the standard FCFS scheduling formulas. Finally, the average Turnaround Time and Waiting Time were computed to evaluate the overall performance of the scheduling algorithm.

The output shows that every process was executed strictly according to its arrival time without any interruption or preemption. This experiment demonstrates that FCFS is one of the simplest scheduling algorithms and is easy to implement. However, if a process with a long burst time arrives before shorter processes, the later processes have to wait longer, which can increase the average waiting time and reduce overall system responsiveness. This limitation is commonly known as the **Convoy Effect**.

Conclusion:

The objective of this experiment was successfully achieved by implementing the FCFS CPU Scheduling algorithm in Python. The program correctly sorted the processes based on their arrival times and calculated the Completion Time, Turnaround Time, and Waiting Time for each process. It also determined the average Turnaround Time and average Waiting Time accurately.

Through this experiment, the basic concept of CPU scheduling and the working principle of FCFS were clearly understood. The results indicate that FCFS is suitable for simple scheduling environments because of its straightforward implementation and fair execution order based on arrival time. However, it is not always the most efficient scheduling algorithm, especially when long processes execute before shorter ones, as this increases the waiting time of subsequent processes. Therefore, although FCFS is easy to understand and implement, more advanced scheduling algorithms such as **Shortest Job First (SJF)** or **Round Robin (RR)** may provide better performance in terms of reducing waiting time and improving CPU utilization in practical operating systems.